https://doi.org/10.1038/s44335-026-00063-7

Improving deep neural network performance through sampling

Lakshmi A. Ghantasala[1,2] ✉, Ming-Che Li[1], Risi Jaiswal[1], Behtash Behin-Aein[2], Joseph Makin[1], Shreyas Sen[1] & Supriyo Datta[1]

Energy-efficient sampling with probabilistic neurons or *p-bits* has been demonstrated in the context of Boltzmann machines, and it is natural to ask if these approaches can be extended to the field of generative AI, where energy costs have become prohibitively large. However, this very active field is dominated by feedforward deep neural networks (DNNs), which primarily use multi-bit deterministic neurons with no role for sampling. In this paper, we first show that it is feasible to obtain superior accuracy through the use of multiple samples generated by probabilistic networks. This possibility raises the question of which option is energetically preferable for improving accuracy: generating more samples or adding more bits to a single deterministic sample. We provide a simple expression that can be used to estimate these energy tradeoffs and illustrate it with results for different algorithms and architectures.

The use of probabilistic neurons or *pbits* comes naturally in the context of Boltzmann machines, where the loss function is expressed in terms of binary stochastic variables, aka classical spins. This has given rise to the vast field of Ising computing, where significant energy advantages have been demonstrated through ASICs designed to solve optimization and sampling problems (see, for example, Li et al. (2025)[1], ISSCC, and references therein). By contrast, the field of generative AI is dominated by feedforward deep neural networks (DNNs), which primarily use multi-bit deterministic neurons with no role for sampling.

Our preliminary results suggest that it may be possible to obtain comparable or even superior accuracy using multiple samples obtained from probabilistic neurons. This possibility immediately raises the question of the energy cost of generating T samples, and the answer depends on various algorithmic and architectural considerations, such as the number of samples in question, the bit precision, and connective fan-out per neuron, among others.

While experimental validation in this work is carried out on FPGA hardware for accessibility and measurement fidelity, the p-DNN architecture is explicitly designed around the p-bit abstraction originally introduced for unconventional computing substrates such as stochastic MTJs, memristive devices, and in-memory compute fabrics. The elementary operation defined in this work mirrors the p-bit synapse-neuron primitive used in prior Ising and Boltzmann machine accelerators, enabling direct mapping onto non-von-Neumann and physics-driven hardware. In this sense, the FPGA serves as a functional emulator of unconventional substrates rather than the target platform.

Our primary objective in this paper is to provide a simple framework that can be used to estimate these energy tradeoffs for different implementations and under different circumstances. We define a universal elementary operation, sketched in Fig. 1a, whose energy cost ϵ_{EO} can be written as (see Fig. 1b)

$$\epsilon_{\mathrm{EO}} = \underbrace{n b_{\mathrm{w}}\,\epsilon_{\mathrm{wM}}}_{\text{read weights}} + \underbrace{(n+1)\,b_{\mathrm{a}}\epsilon_{\mathrm{aM}}}_{\text{read act.s and write out}} + \underbrace{\epsilon_{\mathrm{S}}(n, b_{\mathrm{a}}, b_{\mathrm{w}})}_{\text{synapse energy}} + \underbrace{\epsilon_{\mathrm{N}}}_{\text{neuron energy}} \tag{1}$$

where ϵ_{wM} and ϵ_{aM} represent the energy to access one bit from the weight memory and the activation memory, respectively, ϵ_{S} is the energy needed to compute the function W over n weights, b_{w} and b_{a} are the bits used to represent weights and activations, respectively, and ϵ_{N} is the energy needed to generate a neuron output based on the output of W. The energy to write the result to activation memory is assumed equal to ϵ_{aM}.

Offhand, one might assume that collecting T samples would cost T times as much energy, but this is not true if the samples are all generated from a single reading of the weights without reloading. In this case, Eq. (1) becomes

$$\epsilon_{\mathrm{EO}} = n b_{\mathrm{w}}\,\epsilon_{\mathrm{wM}} + T[(n+1)\,b_{\mathrm{a}}\epsilon_{\mathrm{aM}} + \epsilon_{\mathrm{S}}(n, b_{\mathrm{a}}, b_{\mathrm{w}}) + \epsilon_{\mathrm{N}}] \tag{2}$$

showing that the energy cost of T samples can be minimal if ϵ_{EO} is dominated by the energy needed to load weights (first term).

We emphasize that we view this as a first step towards evaluating the possible role of probabilistic sampling in the machine learning space. Much

[1]Elmore School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN, USA. [2]Ludwig Computing, Mill Valley, CA, USA.
✉e-mail: alasghantasala@gmail.com

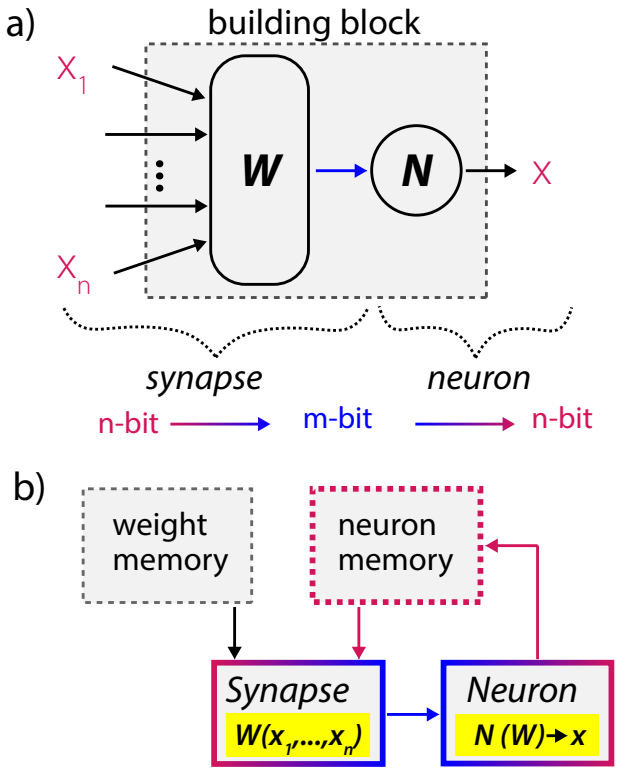

Fig. 1 | The fundamental probabilistic building block and its core operations. **a** Building block of p-circuits, **b** System architecture used to implement the building block, based on which the energy ε per elementary operation (Eq. (1)) is written.

depends on how many samples (T) are needed to achieve a significant enhancement in accuracy. The answer will clearly depend on the algorithm used to generate the samples, where the possibilities are fairly open-ended at this time. Each algorithm will then need to be evaluated with an end-to-end evaluation framework to estimate its energy cost. Only then can we answer the question of whether, for a given performance quality and energy cost, it is preferable to use a deterministic $n-bit$ DNN or an $m-bit$ DNN ($m<n$) with T samples. All we provide in this paper is (1) proof-of-concept that small values of T can lead to significant improvements in performance, and (2) a framework for evaluating energy costs (Eq. (2)) with illustrative examples.

Results

P-bits for DNNs: P-DNNs

Let us first discuss *two possible ways* to incorporate p-bits into deep neural networks (DNNs) to construct probabilistic or p-DNNs.

Sample-aware training of DNNs. A standard approach to realize low-bit DNNs is through *quantization − aware* training, which generally is applied to weights, though the idea can extend to activations as well[2]. We follow a similar approach, but instead train with multiple samples, following a *sample-aware training* scheme as shown in Fig. 2c. This approach involves using the loss value computed from the average of s samples to perform the training by backpropagation[3]. The idea then is to *replace* the activations of a traditional DNN with stochastic activations and then train the model to take into account the stochasticity.

Through sample-aware training, the multiply-accumulate operation, traditionally a computation bottleneck in DNNs[4], is simplified to being between a multi-bit digital weight and a single-bit activation. This dramatically reduces the compute energy, where compute is the sum of synapse and neuron energies, though one has to be wary that drawing multiple samples will again increase that compute energy. We will show that 2 samples is enough to improve accuracy.

We apply this sample-aware training scheme to two p-DNNs, a custom VGG5 model (see Fig. 9b) for image classification trained on the CIFAR10 dataset, and a custom VAE model (see Fig. 9a) for image generation trained on the CelebA dataset.

Image classification: We show that one sample of a 1-bit activation p-DNN is enough to match deterministic accuracy, with 2 samples outperforming. Since this amounts to adding a minor compute energy (and *not* memory energy), the overall energy cost is quite small for the accuracy bump, as seen in Fig. 6b. Additional samples from the p-DNN model lead to higher accuracy, roughly matching 3-bit deterministic accuracy at 10 samples as seen in Fig. 4a. The colored lines in Fig. 4a use multi-sample training and inference by averaging *at the end* of the model. The dotted black line shows averaging test-time samples *at each layer* in the model, with the same weights as those in the deterministic baseline. Interestingly, while the algorithm improves accuracy even at 1-sample inference of a model trained at 45 samples for the classification task, it only improves inference at the same number of samples for the generation task, which we will discuss next.

Image generation: As the baseline, we use a 12-layer Variational Autoencoder DNN using 32-bit weights and sigmoid activations trained to generate celebrity images from a random input (Fig. 3a). Replacing the 32-bit activations with p-bits we get very noisy images, but averaging over 100 samples creates a hint of recognizable facial images (Fig. 3b). Retraining the weights using the actual nonlinear element instead of the continuous version, the quality of images improves significantly (Fig. 3c). Replacing the p-bit activations in the final layer with continuous sigmoid activations marks an improvement, even at 1 sample (Fig. 3d, top row). High-quality images comparable to the original baseline are obtained (see final row of Fig. 3d) with sample-aware training. Figure 3 compares the image quality for different schemes quantitatively using the Fréchet-Inception Distance (FID). A key point to note is that better performance is obtained when the number of samples used in testing *matches* that used in re-training. Furthermore, with sample-aware training, images generated with p-bits can reach very close to 32b FID scores.

Surprisingly, even simple averages at each p-bit output using weights from a 32b deterministic model seem to produce recognizable face images at just 100 samples, as opposed to the expected $O(2^{64})$. It would seem that though standard error applies for *each operation*, it doesn't necessarily apply to the *end application* (e.g. image generation), which has some robustness built into it through the training process. This extends to classification problems as well, as we'll discuss next.

Implementation details for both experiments are included in the appendix.

Sampling noisy DNNs. While sample-aware training requires re-training a model with stochastic activations, there may be ways to take advantage of sampling on DNNs even without retraining, through added noise. By adding some noise to a deterministically trained model, it is possible to generate "noisy samples" that, when averaged, produce lower loss results than a standard model.

We apply this approach to the activations of a 32b model trained for CIFAR-10 classification. The model is run at 4b in a standard post-training quantization fashion[5], with group size 64, as well as quantized after adding some noise to the original model. With just two samples, the sampled model produces better results, shown in Fig. 4c. The improvement shown here is modest, but what is surprising is that the improvement is monotonic. One might have expected that adding noise to a deterministically trained model would randomly improve or degrade performance with samples, but that does not seem to be the case.

We've now discussed two potential approaches to taking advantage of sampling via p-DNNs: a model trained with p-bit activations, and a model with noise added to standard activations. In problems of this type, our aim is not to recreate the activations A precisely but rather to minimize the loss function $\mathcal{L}$ characterizing the mapping of probability distributions. This is facilitated by the fact that, at the optimum, the partial derivatives $\frac{\partial \mathcal{L}}{\partial A}$ are on average zero, so the loss is insensitive to small perturbations in the

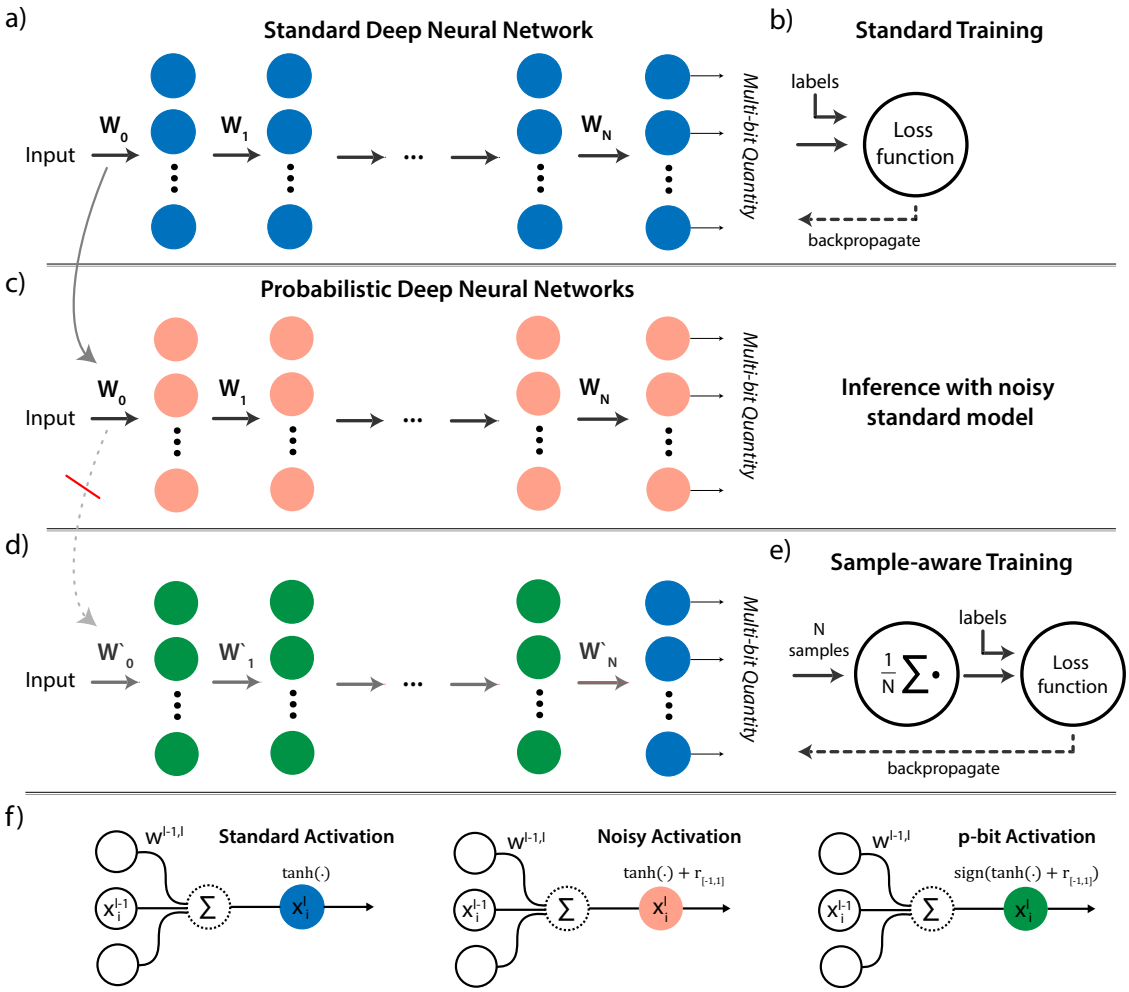

Fig. 2 | Comparison of architectures and training schemes for standard deterministic and probabilistic deep neural networks (p-DNNs). **a** Standard DNNs are implemented with activation functions like tanh or ReLU, and **b** are generally trained following backpropagation. PDNNs can come in different forms, broadly referring to DNNs with stochasticity infused into each forward pass. **c** Shows a model that integrates stochasticity via noisy activations, while **d** shows a model that replaces the activations with pbits. **e** Shows the sample-aware training scheme that dramatically improves performance for pDNNs. **f** Specifies the activation functions.

activations. We can get better results by taking into account more samples to recover any loss of $\mathcal{L}$ due to the noise we add to A. In the next section, we will discuss the energy cost of each implementation and how they (and potentially any other method) fit into the energy framework outlined by Eq. (1).

Contrast with stochastic bitstreaming. It should be noted that using p-bits in place of continuous neurons as described above is *very* different from the well-known use of stochastic bitstreams to represent continuous signals. With stochastic bitstreams, the aim is to generate streams of single bits whose mean value matches the original 32-bit value. Since the mean of a non-linear function is not equal to the function of the mean, sample averaging should be performed individually for *each linear operation at each layer*. Incidentally, we note that the stochastic bitstreaming approach does achieve results that exceed statistical expectations derived from its standard error scaling, though not enough to match the presented mechanisms.

Energy analysis of sampling systems

The solution could be reached faster at the same power level if the energy per elementary operation ϵ_{EO} could be lowered, for example, through the use of in-memory computing and/or analog addition followed by a sampler that uses the analog sum to generate a binary output. These are part of the hardware design, while the quantities N, n and T are set by algorithmic considerations. Let us elaborate a little on the different energy components of the elementary operation described by Eq. (1):

The core components. *Memory access energy*, ϵ_{wM}, ϵ_{aM}: Most large-scale applications in machine learning are memory bottlenecked in latency and energy, making this component especially valuable in an end-to-end analysis of probabilistic solutions. In ref. 1, both weights and p-bit values were stored in local registers, making their read energies identical and relatively low in the tens of fJ range. With large problems, it may not be possible to store all weights in registers, requiring SRAM or even DRAM with much larger read energies. Separating weight and activation memory is useful in that p-bits as activations are binary, and may be stored in closer-to-compute memory than weights, which may be orders of magnitude larger overall. In that case, in-memory computing options can be employed to minimize read energies, which can otherwise dominate the other components (Fig. 6). In Eq. (1), memory access energy involves reading n weights, reading n activations, and writing the output activation of the building block back to memory.

Synapse energy, $\epsilon_S(n, b_w, b_a)$: The function W is usually linear: $w_1x_1 + w_2x_2 + \cdots + w_nx_n$ for b_w-bit w_i quantities. Usually, a multiply-accumulate unit (MAC) is used to evaluate W, but for p-bit activations, since $x_1, \cdots x_n$ become binary variables, the function W only requires an accumulate (AC) function that selectively sums a subset of the weights $w_1, \cdots w_n$. Based on n, there may be